Lab 10: K-Nearest Neighbors & Evaluation Metrics

Learning Objectives

1. Supervised Learning Algorithms
2. K-Nearest Neighbor
3. Evaluation Metrics

---

1. K-Nearest Neighbor

The k-nearest neighbors (KNN) algorithm is a simple, easy-to-implement supervised machine learning algorithm that can be used to solve both classification and regression problems. The K-nearest neighbor algorithm assumes that similar things exist in close proximity to each other. It uses feature similarity for predicting values of new data points. The algorithm helps grouping similar data points together according to their proximity. The main goal of the algorithm is to determine how likely it is for a data point to be a part of the specific group. However, it is more widely used in classification problems in the industry.

In other words, similar things are near to each other (and this is true in most cases). But before implementing k-NN, one must know the optimal value of k (number of nearest neighbors for prediction) to start.
Value of k is selected such that it is neither too small nor too big.

- As we decrease the value of K to 1, our predictions become less stable (Outlier - ???)
- Inversely, as we increase the value of K, our predictions become more stable due to majority voting/averaging, and thus, more likely to make more accurate predictions (up to a certain point).

In classification problems, we usually make K an odd number to have a tiebreaker.

2. Evaluation Metrics

3. Accuracy

Accuracy is the ratio of correct predictions to the total number of predictions. It is used to find the portion of correctly classified values. It tells us how often our classifier is right. It is one of the simplest measures of a model. We must aim for high accuracy for our model. If a model has high accuracy, we can infer that the model makes correct predictions most of the time

$$\text{Accuracy} = \frac{a+d}{a+b+c+d} = \frac{TP+TN}{TP+TN+FP+FN}$$

Without Sklearn (Formula):

```
accuracy = (TP + TN)/(TP + FP + TN + FN)
print(accuracy*100)
```

With Sklearn:

```
from sklearn.metrics import accuracy_score
print(accuracy_score(labels , predictions)*100)
```

4. Recall

➔ Out of all actual positives, how many did we correctly detect?

Without Sklearn (Formula):

```
recall = (TP)/(TP+FN)
print(recall*100)
```

With Sklearn:

```
from sklearn.metrics import recall_score
print(recall_score(labels,predictions))
```

5. Precision

➔ Out of all predicted positives, how many were actually correct?

Without Sklearn (Formula):

```
precision = TP/(TP+FP)
print(precision)
```

With Sklearn:

```
from sklearn.metrics import precision_score
print(precision_score(labels,predictions)*100)
```

6. F1 Score

This is a weighted average of the true positive rate (recall) and precision. F1 score depends on both the Recall and Precision, it is harmonic mean of both the values. It is useful when you need to take both precision and recall into account. It is maximum when Precision is equal to Recall.

Without Sklearn:

```
f1 = 2*(precision * recall)/(precision + recall)
print(f1)
```

With Sklearn:

```
from sklearn.metrics import f1_score
print(f1_score(labels, predictions))
```

Lab Tasks

1. Use the **Iris dataset** uploaded on the Google Classroom.
2. Load it into a Pandas DataFrame and display the first few rows. **(Don't directly load from sklearn.dataset rather use read_csv())**
3. Perform the following Tasks as Demonstrated in Lab.

1. Data Preprocessing

- Load the **Iris dataset** and check for missing values.
- Split the dataset into **80% training and 20% testing**.
- Standardize the features using **StandardScaler**.

2. Train K-Nearest Neighbors (KNN) Model

- Train a **K-Nearest Neighbors classifier** with the default parameters (use `KNeighborsClassifier` from `sklearn`).
- Evaluate the model's performance using accuracy.
- Experiment with different values of `k` (e.g., 3, 5, 7) and analyze the performance.

3. Model Evaluation

- Compute and print the following metrics for KNN:
  - **Accuracy**
  - **Precision**
  - **Recall**
  - **F1-Score**
  - **Confusion Matrix**

4. Visualization

- Plot the **Confusion Matrix** for both models to evaluate performance.

4. Go through the attached Colab Notebook for reference.

Lab # 10 Marks distribution

	ER1	ER5	ER8
Task	3 points	3 points	4 points

Lab # 10 Rubric Evaluation Guideline:

#	Qualities & Criteria	0 < Poor <= 1	1 < Satisfactory <= 2	2 < Excellent <=3
ER 1	**Task Completion**	Minimal or no program functionality was achieved.	Some tasks were completed, but the program has errors or incomplete functionalities.	All tasks were completed, and the program runs without errors.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER 5	**Analysis of Results**	No attempt to analyze or interpret the results	Some analysis of the results is attempted, but explanations may be unclear or incomplete.	Results are correctly analyzed and compared to expectations, with clear explanations.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1.5 < Satisfactory <= 2.5**	**2.5 < Excellent <= 4**
ER 8	**Question & Answer**	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.